

Version 1.2

August 2026



MACHINE LEARNING

MODULE 1 - INTRODUCTION TO ARTIFICIAL NEURAL NETWORKS

Authored by:

Ir. Yufis Azhar, S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY OF MUHAMMADIYAH MALANG

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
INTRODUCTION.....	3
Practical Objectives.....	3
Module Target.....	3
Preparation.....	3
MAIN TOPICS.....	4
A. ARTIFICIAL NEURAL NETWORK (ANN).....	4
B. BASIC STRUCTURE OF ARTIFICIAL NEURAL NETWORK (ANN).....	4
C. TRAINING PROCESS.....	5
D. BATCH SIZE AND EPOCH.....	6
E. TYPES OF ARTIFICIAL NEURAL NETWORKS.....	7
SIMPLE SIMULATION.....	8
CODELAB (10%).....	13
LAB ASSIGNMENT (90%).....	16



INTRODUCTION

Practical Objectives

This module aims to give students a basic understanding of Artificial Neural Networks for both technical and non-technical backgrounds. Students will understand the basic concepts of Artificial Neural Networks (ANN), their structure and types, and how they are used in various contexts.

Module Target

Students can understand the module concepts, create a simple Artificial Neural Network model for binary classification with tabular datasets, and complete the practical module activities.

Preparation

1. Muslim students may recite the following prayer before starting the lesson

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Google Collaboratory.
3. Library NumPy, Pandas, Matplotlib, Sklearn, dan Tensorflow.
4. Source Code :

∞ MODULE 1 MACHINE LEARNING - INTRODUCTION TO ARTIFICIAL NEURAL NETWORK



MAIN TOPICS

Why should we study Artificial Neural Networks? Artificial Neural Networks are the foundation of many artificial intelligence technologies we use every day, such as face recognition, recommendation systems, automatic language translators, and others. Artificial Neural Networks have an extraordinary ability to recognize patterns from complex data.

A. ARTIFICIAL NEURAL NETWORK (ANN)

An Artificial Neural Network, or neural network, is a machine learning model that helps computers make decisions using a method similar to the human brain. This process mimics how biological neurons work together to identify patterns, consider options, and reach a decision or conclusion.

B. BASIC STRUCTURE OF ARTIFICIAL NEURAL NETWORK (ANN)

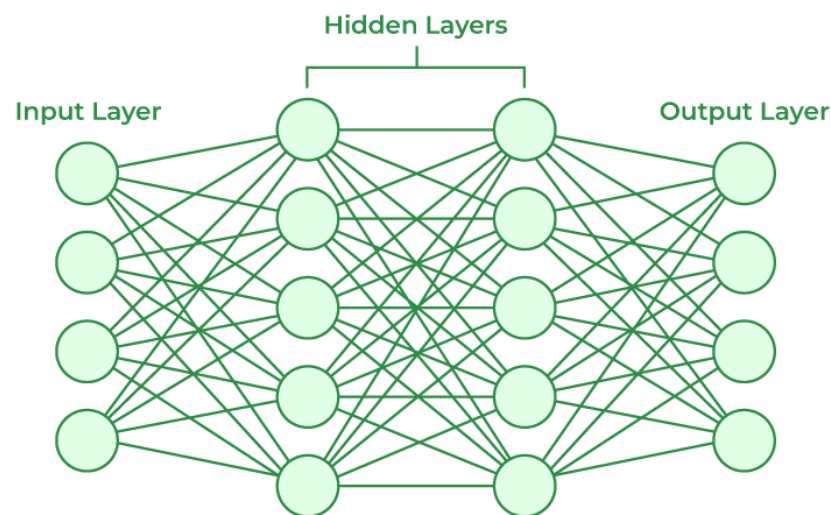


Figure 1. Visual representation of an epoch divided into batch sizes

Source: [Artificial Neural Networks and its Applications - GeeksforGeeks](https://www.geeksforgeeks.org/artificial-neural-networks-and-its-applications/)

Artificial Neural Networks consist of several layers, with three main types:

1. **Input layer:** This layer receives raw data or features from the dataset. For face recognition, the input can be a face image.
2. **Hidden layer:** This layer processes data from the input layer. The more hidden layers there are, the more complex the patterns the network can learn. Each hidden layer transforms the information into more complex, abstract representations.



3. **Output layer:** This layer will produce the final decision or prediction. The output layer translates the information processed by the hidden layer into an understandable and usable form. For example, after processing an image, the output layer will decide whether the result is a cat image or a dog image.

C. TRAINING PROCESS

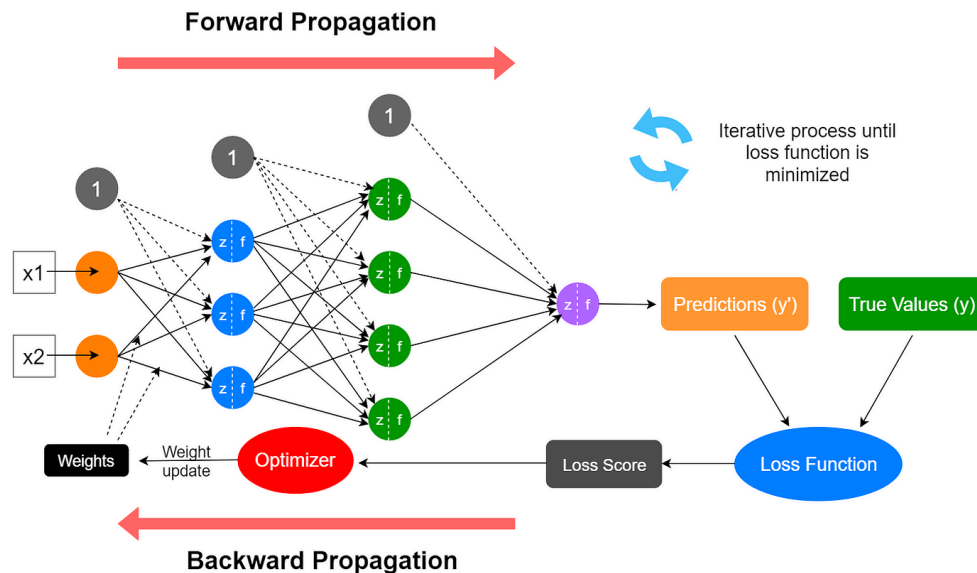


Figure 2. The Process of Forward and Backward Propagation in Artificial Neural Networks

Source: [Overview of a Neural Network's Learning Process | by Rukshan Pramoditha | Data Science 365 | Medium](#)

The training (learning) process of an artificial neural network is iterative: calculations are performed forward and backward through each layer until the loss function is minimized. The entire series of processes is divided into three parts:

1. Forward Propagation

Artificial Neural Networks are made up of several neurons stacked into several layers. These layers are connected by network parameters (represented by arrows). These parameters are weights and biases. Weights control the importance of each input. Biases determine how easily neurons activate.

First, initialize the network parameters with random, non-zero values for weights and biases. Based on the input values and the initialized values, the neuron calculates its linear function and activation function. This calculation occurs throughout the network and produces the final result of forward propagation in the first iteration.



In forward propagation, calculations are performed from the input layer to the output layer (left to right) through the network.

2. Loss Function Calculation

The output of forward propagation is called the predicted value. This value must be compared with the real value to measure the artificial neural network's performance. This is where the loss function (also called the objective or cost function) comes in.

The loss function calculates the loss score between the predicted value and the actual value. This is also known as the model error. The loss function measures how well the model performs at each iteration. The loss function provides feedback for parameter updates during backpropagation. The ideal loss value is 0. The closer it is to 0, the better the model's predictions.

3. Backward Propagation

In the first iteration, the predicted value is far from the true value. This is because the parameters (weights and biases) are set arbitrarily. These values are not optimal. Therefore, we must update the parameters to minimize the loss function. The parameter update process uses an optimizer that applies backpropagation. The optimizer aims to find the loss function's global minimum.

In backpropagation, the chain rule computes the gradient of the loss function with respect to the model parameters. This gradient indicates the direction to update the model parameters. Keras (a Python library) automatically calculates these derivatives through automatic differentiation. Calculations are performed from the output layer to the input layer (from right to left).

D. BATCH SIZE AND EPOCH

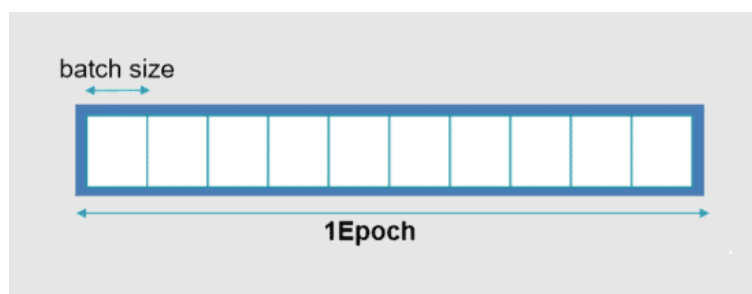


Figure 3. Visual Illustration of One Epoch Partitioning by Batch Size

Source: [Epoch: An essential notion in real-time programming](#)

When dealing with the training and testing process, there are generally parameters that we can tune (adjust), including epoch and batch size.



Batch size is the number of data samples that typically pass through a neural network at one time. Batch size determines how many samples the model processes before updating its internal parameters. For example, we have a dataset containing 1 million image samples for training. Although the neural network we use is not too complex, 1 million training data points are still too many to process simultaneously.

An epoch is a hyperparameter that determines how many times the algorithm works through the entire dataset both forward and backward. One epoch is reached when all batches have successfully passed through the neural network once.

E. TYPES OF ARTIFICIAL NEURAL NETWORKS

There are various types of Artificial Neural Network (ANN) architectures, including:

1. **Feedforward Neural Network (FNN):** The simplest type of artificial neural network where information flows in one direction from the input layer, through the hidden layer, and to the output layer. This network has no loops or feedback.
2. **Convolutional Neural Network (CNN):** Specifically designed for processing grid data such as images. CNNs use convolutional layers that can recognize complex visual patterns by applying filters to the input.
3. **Recurrent Neural Network (RNN):** Has recurrent connections that allow information to persist along a sequence of data. This makes RNNs useful for data with temporal or sequential dependencies.
4. **Long Short-Term Memory (LSTM):** A special type of RNN designed to overcome the vanishing gradient problem by using memory cells that can retain information for longer periods.



SIMPLE SIMULATION

Let's create a simple simulation of an Artificial Neural Network (ANN) for a binary classification task using Python and TensorFlow/Keras. Here are the steps:

1. Data Preparation

We will use a simple dataset like `make_moons` from `sklearn` to create easily understandable data. First, we import the necessary libraries.

```
# Import library yang digunakan
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

Next, we use the `make_moons` function from the `sklearn.datasets` library to generate a synthetic dataset commonly used for classification problems.

```
# Generate dataset
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
```

Then, we split the data into training and test data.

```
# Split dataset menjadi training dan testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)
```

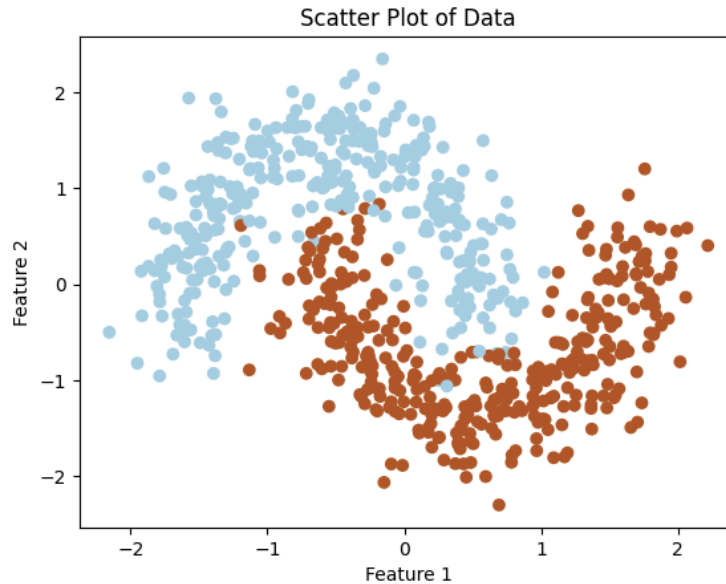
Next, perform sufficient preprocessing. In this simulation, we apply normalization using "StandardScaler" to ensure the features have a similar scale, which can improve the performance of machine learning algorithms.

```
# Standarisasi data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Finally, before building the Model, let's look at the data distribution using a Scatter Plot.

```
# Plotting data
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=plt.cm.Paired)
plt.title('Scatter Plot of Data')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```





2. Building the Model

The model used in this simulation is a simple Feedforward Neural Network (FNN) built using the Keras framework from TensorFlow. First, let's import the libraries we need.

```
# Import library yang akan digunakan
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Next, let's build the model.

```
# Membangun model
model = Sequential([
    Dense(10, input_shape=(2,), activation='relu'),
    Dense(10, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

This model consists of three layers: an input layer with 10 neurons that receives two input features and uses the ReLU activation function, a hidden layer with 10 neurons that also uses the ReLU activation function, and an output layer with 1 neuron that uses the sigmoid activation function for binary classification. This model classifies data into two classes based on its input features. After creating the model, compile it by defining the optimizer, loss, and metrics parameters.

```
# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```



Here is an explanation of each of the parameters used:

- Optimizer:** The optimizer function is an algorithm that adjusts and updates your model's parameters during training. Its goal is to minimize the loss function by adjusting the model's weights and biases. Popular optimizers in machine learning include SGD (Stochastic Gradient Descent), Adam, RMSprop, and more. Each optimizer has its own advantages and disadvantages and may suit certain types of problems and data.
- Loss:** The loss function measures how well or poorly your model predicts the training data compared to the actual labels. Its goal is to measure the size of the model's prediction error. Choose a loss function that fits the problem you are solving. For example, for classification problems, you can use Cross-Entropy Loss, while for regression problems, you can use Mean Squared Error (MSE) Loss. The right choice will help your model learn better.
- Metrics:** Metrics are measurements used to evaluate model performance after training. These metrics provide additional information beyond the loss function and help you understand how well your model has solved the problem. Common metrics include accuracy for classification problems, Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE) for regression problems, and others such as F1-score, Precision, Recall, etc., depending on the problem type.

After the model has been successfully compiled, it is ready to be trained. Before entering the training stage, let's display a summary of the model we have created.

3. Training the Model

In this stage, we will train the model using the training data created previously. The `model.fit` function requires input data `x_train` and labels `y_train`, and trains the model for 100 epochs. We use 20% of the training data as validation data to monitor performance and help prevent overfitting. The training results, including performance metrics such as accuracy and loss for each epoch, are stored in the `history` object for further analysis and visualization.

```
# Train Model
history = model.fit(X_train, y_train, epochs=100, validation_split=0.2)
```

4. Model Evaluation

After training, we evaluate the model to determine its performance.

```
# Evaluasi Model
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Loss: {loss:.4f}')
print(f'Test Accuracy: {accuracy:.4f}')
```



```
10/10 ————— 0s 8ms/step - accuracy: 0.9822 - loss: 0.0741
Test Loss: 0.0850
Test Accuracy: 0.9700
```

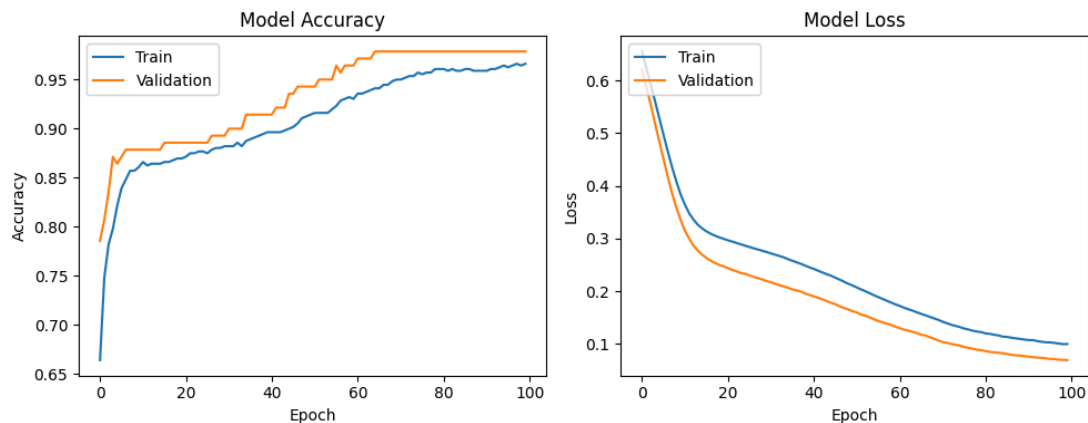
We can also display accuracy and loss graphs for training and validation data during training, as follows.

```
plt.figure(figsize=(10,4))

# Plot akurasi
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
plt.show()
```



Also display its Classification Report as follows.

```
from sklearn.metrics import classification_report

# Prediksi pada data test
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
print(classification_report(y_test, y_pred_classes))
```



10/10			0s 5ms/step		
		precision	recall	f1-score	support
	0	0.96	0.98	0.97	156
	1	0.98	0.96	0.97	144
	accuracy			0.97	300
	macro avg	0.97	0.97	0.97	300
	weighted avg	0.97	0.97	0.97	300

Test the model on new data.

```
# Data baru untuk diuji (contoh 2 titik data)
X_new = np.array([[0.5, 0.3], [1.2, -0.8]])
proba = model.predict(scaler.transform(X_new))
kelas = (proba > 0.5).astype(int)

for d, p, k in zip(X_new, proba, kelas):
    print(f>Data: {d}, Prob kelas 1: {p[0]:.4f}, Prediksi: {k[0]}")
```

```
1/1 0s 42ms/step
Data: [0.5 0.3], Prob kelas 1: 0.3757, Prediksi: 0
Data: [ 1.2 -0.8], Prob kelas 1: 0.9987, Prediksi: 1
```

Congratulations! With this, we have successfully created a simple binary classification model.



CODELAB (10%)

Complete the code below based on the instructions provided!

Dataset: [CODELAB DATASET](#)

1. Import Necessary Libraries. (10 Points)

```
import ... as np
import ... as pd
import ... as plt
from ... import MinMaxScaler

from tensorflow.keras.layers import ...
from tensorflow.keras.models import ...
from sklearn.model_selection import ...
```

2. Load Dataset. (10 Points)

```
pd.read_csv('...')

# Tampilkan 10 data awal
...

# Tampilkan informasi ringkas data
...

# Tampilkan statistik deskriptif data
...
```

3. Data Preprocessing. (10 Points)

```
# Drop kolom yang tidak diperlukan
...(columns=['customer name', 'customer e-mail', 'country', 'gender'], inplace=True)

# Definisikan X dan y
X = df.drop('car purchase amount',axis=1)
y = df['car purchase amount']
```

4. Data Standardization. (10 Points)

```
scaler = ...
X = scaler...(X)
y = scaler...(y.values.reshape(-1, 1))
```



5. Data Splitting. (10 Points)

Gunakan test size 20% dan random state 0

```
X_train, X_test, y_train, y_test = train_test_split(..., ..., ..., ...)

print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

6. Build Model. (10 Points)

Conditions:

- Use a Sequential model
- There are 3 layers of Dense type
- The first layer has 10 neurons, ReLU activation
- The second layer has 10 neurons, ReLU activation
- The third layer has 1 neuron, linear activation
- Compile the model using the Adam optimizer and mean_squared_error loss function

```
model = ...([
    ...(..., activation='...', input_dim=4),
    ...(..., activation='...'),
    ...(..., activation='...')
])

model.compile(optimizer='...', loss='...')
model.summary()
```

7. Training Model. (10 Points)

- Use 100 epochs
- 20% validation data

```
history = model.fit(X_train, y_train, epochs=..., validation_split=...)
```

8. Model Evaluation. (10 Points)

```
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Loss: {loss:.4f}')
print(f'Test Accuracy: {accuracy:.4f}')
```

9. Plotting Accuracy and Loss from Training and Validation. (10 Points)

Create all the code yourself.



DETAILS OF CODELAB'S WORK

DETAIL	POINT
Import Necessary Libraries	10
Load Dataset	10
Data Preprocessing	10
Data Standardization	10
Data Splitting	10
Build Model	10
Training Model	10
Model Evaluation	10
Plotting Accuracy and Loss from Training and Validation	20
TOTAL	100



LAB ASSIGNMENT (90%)

1. Import Library dan Load Data (5%)

- Import all necessary libraries. (30 points)
- Load the dataset from the following link: [\[ASSIGNMENT DATASET\]](#) (70 points)

2. Exploratory Data Analysis (5%)

- Display the top 5 and bottom 5 rows of the dataset. (30 points)
- Display a statistical summary of the dataset, including mean, median, standard deviation, minimum value, maximum value, and quartiles. (35 points)
- Visualize the label distribution with a graph. (35 points)

3. Data Preprocessing (5%)

Handle missing values with the following conditions:

- Fill empty values in the 'IPK' and 'Kehadiran_Persen' columns with the mean. (30 points)
- Fill empty values in the 'Tugas_Persen' column with the median. (35 points)
- Fill empty values in the 'Nama' column with 'Nama_Tidak_Diketahui'. (35 points)

4. Initialize Input (5%)

Separate data into features and targets. (100 points)

5. Splitting Data with 80:20 Proportion (5%)

Perform data splitting with an 80:20 split ratio. (100 points)

6. Build Naive Bayes Model (10%)

Build a model for testing using Naive Bayes. (100 points)

7. Evaluate the Model with the Following Conditions: (5%)

- Display the classification report along with label descriptions. (50 points)
- Minimum accuracy of 85%. (50 points)

8. Test with New Data: (5%)

- GPA 3.2, Attendance_Percentage 85%, Assignment_Percentage 20%. (30 points)
- GPA 2.1, Attendance_Percentage 60%, Assignment_Percentage 50%. (35 points)
- GPA 3.8, Attendance_Percentage 95%, Assignment_Percentage 90%. (35 points)

Expected Output:

```
Tidak Lulus  
Tidak Lulus  
Lulus
```



FINAL GRADE WEIGHTING

Activity	Description	Weight
Codelab		10%
Lab Assignment	Import Library dan Load Dataset	5%
	Exploratory Data Analysis	5%
	Data Preprocessing	5%
	Initialize Input	5%
	Data Splitting	5%
	Build Model	10%
	Model Evaluation	5%
	Test with New Data	5%
Explanation to Assistant		45%
Total		100%

